



UNIVERSIDADE DO MINHO
LICENCIATURA EM ENGENHARIA INFORMÁTICA

Comunicação de Computadores

Trabalho Prático 2:

Protocolos da Camada de Transporte

David Alves, Gonçalo Amaro, Tomás Ramalhete

Turno PL2 - Grupo 22

Universidade do Minho

e-mail: {a107324,a106803,a98953}@alunos.uminho.pt

1. Introdução

Este relatório apresenta o desenvolvimento de um sistema de comunicação espacial simulado, concebido no âmbito da unidade curricular de Comunicações por Computador da Licenciatura em Engenharia Informática (Universidade do Minho, 2025/2026). O projeto foca-se na implementação de dois protocolos aplicacionais - MissionLink (ML) e TelemetryStream (TS) - que exploram as camadas de transporte UDP e TCP, respetivamente, para garantir comunicação eficiente e fiável entre uma Nave-Mãe e múltiplos rovers numa missão espacial simulada.

2. Arquitetura Implementada da Solução

A solução segue uma arquitetura distribuída do tipo cliente-servidor, onde a Nave-Mãe atua como o servidor central de comando e controlo, e os Rovers atuam como clientes autónomos que operam na superfície planetária.

A comunicação entre as entidades é assegurada por dois canais distintos, otimizados para propósitos diferentes: MissionLink (UDP) para transações críticas de missão e TelemetryStream (TCP) para monitorização contínua.

2.1. Processo de Sincronização Inicial (Service Discovery)

O sistema não requer configuração manual de IDs nos Rovers. O processo de arranque segue a seguinte lógica:

1. A Nave-Mãe é iniciada primeiro e carrega uma tabela de mapeamento estático (IP para ID).
2. Os Rovers iniciam sem identidade definida e enviam um pedido de registo (Req) para o IP conhecido da Nave-Mãe via UDP (Porta 50000).
3. A Nave-Mãe identifica o Rover através do seu endereço IP de origem, regista a sua presença na lista de rovers ativos (rovers_info.json) e responde com um ACK contendo o ID atribuído (ex: "A02").
4. O mecanismo implementa backoff exponencial. Se o Rover não receber o ACK, duplica o tempo de espera a cada tentativa (2s, 4s, 8s...), até um limite

de 5 tentativas, garantindo que o sistema não entra em loop infinito em caso de falha de rede.

2.2. Operação Autônoma

Após a sincronização, os Rovers entram em modo autônomo. A comunicação é iniciada pelo Rover, que pede missões (MReq) quando se encontra no estado "parado". Durante a execução, o Rover recolhe dados de sensores simulados e gere a sua própria bateria, reportando o estado à Nave-Mãe.

2.3. Protocolos de Comunicação

A solução implementa dois protocolos distintos para garantir eficiência e fiabilidade:

- **MissionLink (UDP - Porta 50001):** Visto que o UDP não garante entrega, foi implementada uma camada de fiabilidade aplicacional (*Reliability Layer*) que inclui números de sequência, confirmações (ACKs), temporizadores de retransmissão e verificação de integridade.
- **TelemetryStream (TCP - Porta 50010):** Aproveita as garantias intrínsecas de entrega ordenada do TCP para o envio contínuo de dados de monitorização.

3. Mission Link (ML)

O Mission Link (ML) é o subsistema responsável pela comunicação crítica entre a Nave-Mãe e os Rovers. O objetivo principal foi desenvolver um protocolo aplicacional robusto sobre UDP (User Datagram Protocol).

Dada a natureza "best-effort" do UDP (que não garante entrega nem ordem), a complexidade da fiabilidade foi movida para a camada de aplicação. O protocolo foi desenhado para gerir a atribuição de missões, monitorização de progresso e gestão de falhas, assegurando que nenhuma ordem crítica é perdida ou duplicada.

3.1. Arquitetura do protocolo

3.1.1. Formato do header

Neste projeto, no mission link, todas as mensagens todas os parâmetros das mensagens foram primeiro convertidos em caracteres (1 caractere ocupa 1 byte) antes de ser, por fim, convertido em bytes.

Todas as mensagens do mission link utilizam este mesmo header.

Campo	Tamanho (Bytes)	Descrição
Rover ID	2	Identificador único do remetente/destinatário (ex: 02).
Seq	3	Número de sequência da mensagem atual.
Ack Seq	3	Número de sequência que está a ser confirmado.
Type	4	Tipo de mensagem (ex: MReq, MHan, MRep).
Timestamp	14	Carimbo temporal (YYYYMMDDHHMMSS).
Payload Size	3	Tamanho exato do corpo da mensagem em bytes.

Tabela 1: Bytes usados para cada componente do header.

Rover ID:

Refere ao ID do rover, quer seja remetente ou destinatário. Apesar de a nossa topologia ter apenas 3 rovers, nós utilizamos 2 bytes para garantir alguma escalabilidade, permitindo usar até 99 rovers.

Seq:

“Seq” é um id da mensagem enviada que é incrementada a cada mensagem diferente enviada, por outras palavras, é um ID atômico e único para cada mensagem diferente. Cada entidade (rovers e nave mãe) tem um “Seq” independente. “Seq” será 0 apenas de a mensagem for um acknowledgment de outra mensagem recebida.

Ack Seq:

Funciona como o parâmetro “Seq” mas apenas para mensagens do tipo “MAck”. Se “Ack Seq” for diferente de 0, Seq tem que ser 0.

Payload Size:

Representa o número de bytes que representam o payload. Utilizado para propósitos de verificação de integridade da mensagem ao ser recebida.

Type:

O “Type” diz ao recetor para que serve a mensagem recebida e o que fazer com ela. Com cada “Type” um payload diferente será usado.

Código	Nome	Direção	Descrição	Conteúdo do Payload
MReq	Mission Request	Rover → Nave-Mãe	O Rover solicita uma nova missão quando está no estado IDLE.	Vazio (0 bytes).
MHan	Mission Handle	Nave-Mãe → Rover	A Nave-Mãe atribui uma missão específica ao Rover.	Missão
MRep	Mission Report	Rover → Nave-Mãe	O Rover envia dados parciais de progresso durante a execução.	Relatório de Missão
MCon	Mission Conclusion	Rover → Nave-Mãe	O Rover informa que a missão foi terminada com sucesso (100%).	Relatório de conclusão de missão.
MAck	Acknowledgment	Ambos	Confirmação de receção de uma mensagem crítica (fiabilidade).	Vazio (0 bytes).

MEnd	End of missions	Nave-Mãe → Rover	A nave mãe reporta que o buffer de missões está vazio, desligando o rover	Vazio (0 bytes).
------	-----------------	---------------------	--	------------------

Tabela 2: Tabela com informações sobre os tipos de mensagens.

3.1.2. Formato dos Payloads

Tal como no cabeçalho, os payloads seguem um formato de largura fixa (Fixed-Width) para garantir previsibilidade e eficiência no processamento. A estrutura interna varia consoante o tipo de tarefa (I: Imagem, S: Solo, A: Atmosfera).

Payload de Atribuição de Missão:

Campo	Tamanho (Bytes)	Descrição
Mission ID	4	Identificador único da missão.
Task Code	1	Tipo de tarefa (I, S ou A).
Coord X	5	Coordenada X do destino.
Coord Y	5	Coordenada Y do destino.
Duração	5	Tempo máximo estimado para a missão (segundos).
Intervalo	3	Periodicidade de envio de relatórios (segundos).
Bat Min	5	Bateria mínima prevista necessária.
Params	Variável	Parâmetros específicos (ex: N° Fotos, Profundidade).

Tabela 3: Informações sobre os componentes do payload de atribuição de missões.

Payload de Relatório/Conclusão:

Campo	Tamanho (Bytes)	Descrição
Mission ID	4	ID da missão a que se refere o relatório.
Progress	3	Percentagem de conclusão (000 a 100).
Task Code	1	Tipo de tarefa executada.
Time	5	tempo de execução
Data	Variável (descrito na tabela abaixo)	Resultados da simulação (ex: temperatura, nome das imagens).

Tabela 4: Informações sobre os componentes do payload de relatórios.

Dados dos relatórios:

Código	Tarefa	Estrutura dos Parâmetros (Bytes)	Exemplo de Codificação
I	Captura Imagens	Num Fotos (2) + Direção (1)	03N (3 fotos, Norte)
S	Coleta Solo	Profundidade (5) + Peso (4)	015.50500 (15.5cm, 500g)
A	Análise Atmosfera	Temp (6) + Gases (3 cada: CO2, O2, N2)	-20.50095005001 (-20.5°C, 95% CO2...)

Tabela 5: Informações sobre o tipo de dados dos relatórios.

3.2. Mecanismo de Fiabilidade

Dado que o protocolo UDP não garante a entrega de pacotes, foi implementado um mecanismo do tipo Stop-and-Wait na camada de aplicação.

3.2.1. Lógica do Cliente (Rover)

Para garantir que mensagens críticas (como pedidos de missão ou relatórios) chegam ao destino:

1. **Retransmissão:** O Rover envia a mensagem e aguarda uma resposta específica (“MAck” ou “MHan” ou “MEnd”) com o “Ack Seq” correspondente.
2. **Timeouts e Backoff:** Se a resposta não chegar dentro de um tempo limite (inicialmente 2 segundos), o Rover assume perda de pacote. O tempo de espera duplica a cada tentativa falhada (*Exponential Backoff*), evitando congestionamento da rede.
3. **Limite de Tentativas:** Após 3 tentativas sem sucesso, o Rover declara erro de comunicação e aborta a operação, prevenindo bloqueios infinitos (“deadlocks”).

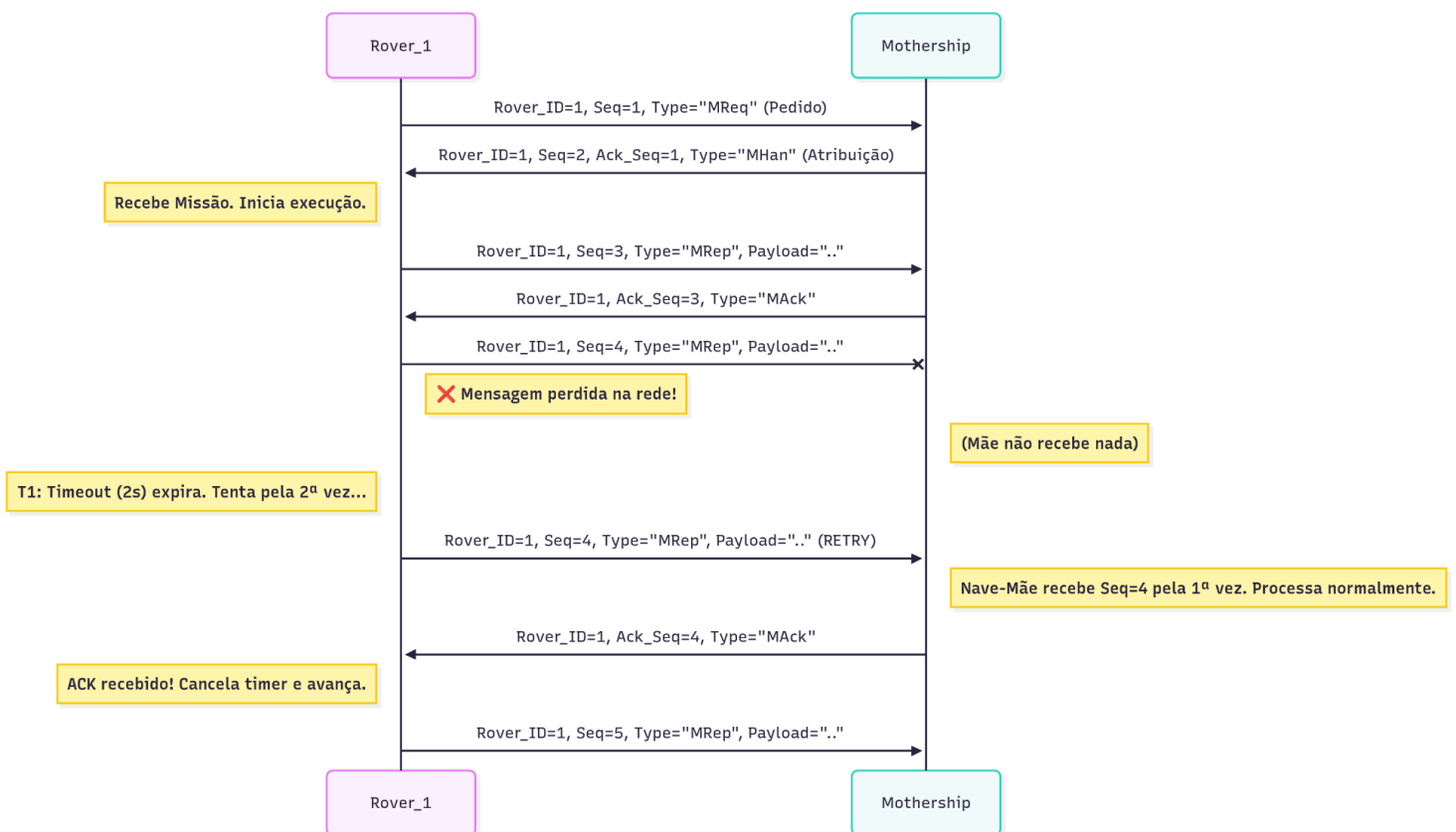


Figura 1: Diagrama “UDP Unhappy path: rover não consegue conectar a Mãe.”

3.2.2. Lógica do Servidor (Nave-Mãe)

A Nave-Mãe implementa um sistema de memória (`reliability_db`) para garantir que processar o mesmo pedido várias vezes não causa erros.

- **Deteção de Duplicados:** A Nave-Mãe armazena o último Seq recebido de cada Rover. Se receber um pacote com um Seq já processado, identifica-o como uma retransmissão (o ACK original perdeu-se).
- **Resposta em Cache:** Em vez de processar a missão ou o relatório novamente, a Nave-Mãe reenvia imediatamente a última resposta gerada, garantindo que o Rover recebe a confirmação necessária para avançar.

3.3. Ciclo de Vida e Gestão de Energia

O subsistema Mission Link atua em coordenação direta com o Simulador de Física e a Telemetria TCP para gerir o ciclo de vida do Rover.

1. **Validação de Energia:** Antes de atribuir uma missão (MReq), a Nave-Mãe verifica via telemetria se a bateria atual do Rover é suficiente para cobrir o custo estimado da missão mais uma margem de segurança. Se não for, envia um MAck de recusa e o Rover aguarda.
2. **Execução Física:** Ao receber uma missão (MHan), o Rover transita o seu estado físico para "a_caminho", deslocando-se virtualmente até às coordenadas alvo antes de iniciar a recolha de dados ("em_missao").
3. **Recuperação de Falhas:** Se a bateria atingir níveis críticos ($< 20\%$) durante a operação, o Mission Link deteta a alteração de estado para `low_power_sleep` (gerado pela física), pausa o envio de relatórios e entra em modo de hibernação até que a recarga automática esteja completa.

3.4. Fluxo de Execução

O funcionamento do sistema segue um ciclo contínuo de estados. Abaixo descreve-se o fluxo detalhado desde o pedido de uma missão até à sua conclusão.

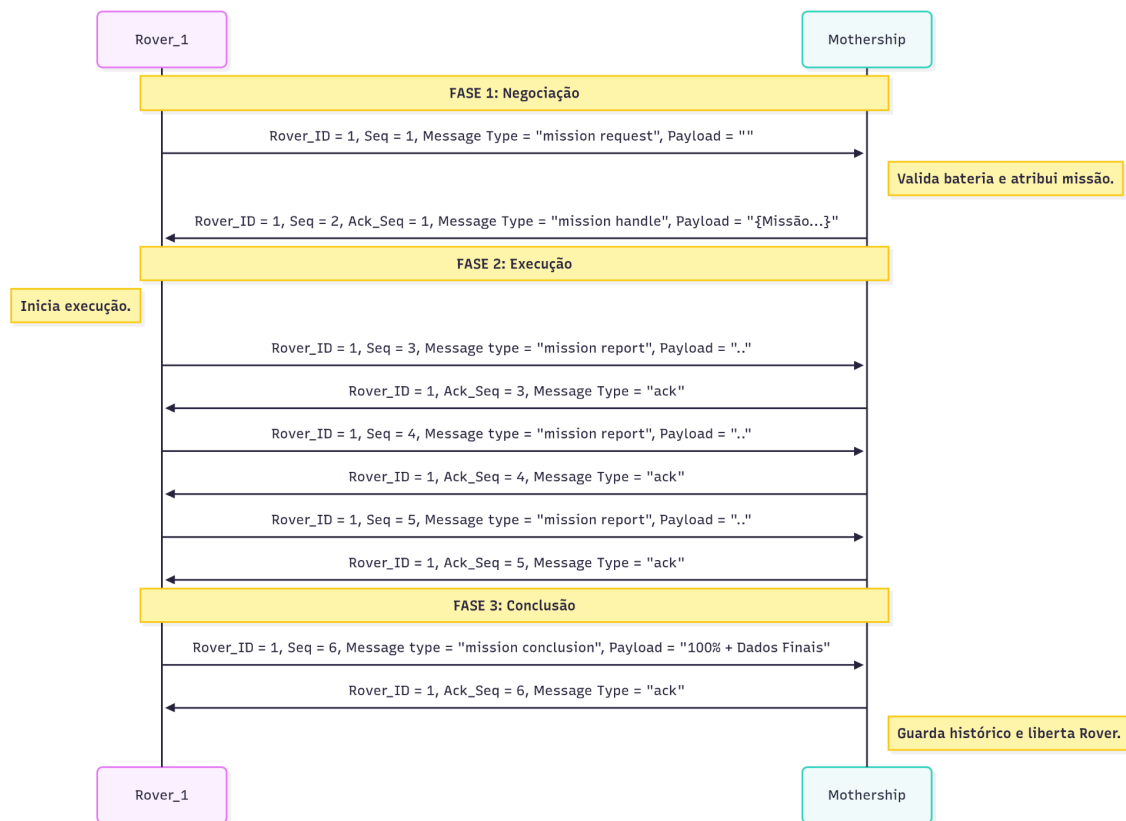


Figura 2: Diagrama “UDP Happy path.”

Fase 1: Negociação

1. **Pedido:** O Rover, estando no estado “parado”, inicia a comunicação enviando um “MReq” (Mission Request) via UDP.
2. **Validação:** A Nave-Mãe recebe o pedido e consulta a Telemetria mais recente do Rover na sua base de dados partilhada.
 - *Cenário A (Rejeição):* Se a bateria for insuficiente, a Nave-Mãe envia um “MAck” vazio. O Rover interpreta isto como uma ordem de espera (“Standby”) e aguarda antes de tentar novamente.
 - *Cenário B (Aceitação):* Se as condições forem ideais, a Nave-Mãe retira a missão prioritária da lista, empacota os dados num payload “MHan” e envia-o ao Rover.

Fase 2: Deslocação

Ao receber o “MHan”, o Rover altera o seu estado interno para “a_caminho”, simulando a deslocação para a localização da missão. O módulo Mission Link

calcula o vetor de deslocação entre a posição atual e as coordenadas alvo da missão. O Rover move-se, atualizando a sua posição (X, Y) em tempo real, o que é refletido na Telemetria TCP enviada paralelamente à Nave-Mãe.

Fase 3: Execução e Monitorização

1. **Execução da missão:** Ao chegar ao destino, o estado muda para “em_missão”. O Rover entra num ciclo de recolha de dados (simulados) conforme a tarefa (Imagens, Solo ou Atmosfera).
2. **Relatórios progressivos:** Periodicamente (conforme o intervalo definido na missão), o Rover envia um “MRep” (Mission Report) com a percentagem de progresso e os dados recolhidos.
3. **Fiabilidade de Dados:** O Rover aguarda a confirmação (“MAck”) de cada relatório. Se a confirmação falhar (timeout), o Rover regista o aviso mas continua a missão, garantindo que a operação não bloqueia por falhas temporárias de rede.

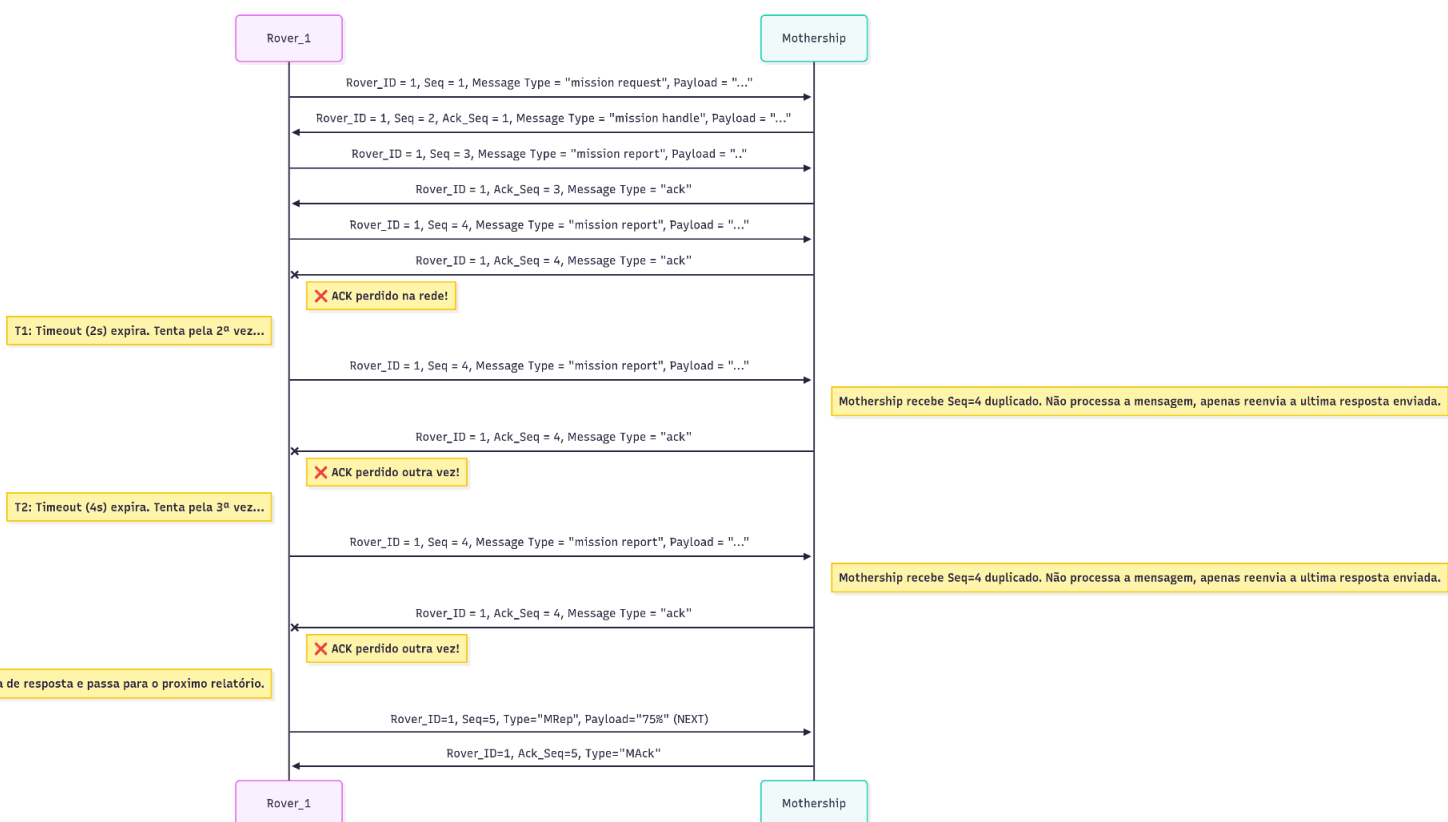


Figura 3: Diagrama “UDP Unhappy path: Mãe não consegue responder a rover.”

Fase 4: Conclusão e Persistência

1. **Finalização:** Ao atingir 100% de progresso, o Rover envia um “MCon” (Mission Conclusion) com os dados finais.
2. **Arquivo:** A Nave-Mãe recebe o “MCon”, guarda o registo completo no ficheiro “completed_missions.json” e liberta o Rover.
3. **Reset:** O Rover volta ao estado “parado”, pronto para iniciar um novo ciclo na Fase 1.

Fase 5: Gestão de Exceções

A qualquer momento durante a Fase 3, se o Simulador de Física detetar que a bateria desceu abaixo dos 20%, força a alteração do estado para “low_power_sleep”.

O Mission Link deteta esta interrupção, pausa o ciclo de progresso da missão e entra em modo de hibernação.

Após a recarga simulada (30s), o estado é restaurado automaticamente e a missão retoma exatamente do ponto onde parou.

4. Telemetria

O subsistema de Telemetria é responsável pela monitorização em tempo real do estado de todos os Rovers ativos. Ao contrário do Mission Link (que é orientado a eventos e transacional), a Telemetria exige um fluxo contínuo e fiável de dados para que a Nave-Mãe possa tomar decisões informadas (como validar a bateria antes de atribuir uma missão) e atualizar a interface de controlo (API).

4.1. Arquitetura e Protocolo

Para otimizar o *parsing* e garantir determinismo no processamento de *streams* contínuos, foi desenhado um protocolo de mensagem com tamanho fixo de 39 bytes.

Estrutura da Mensagem de Telemetria

Nome	Tamanho (Bytes)	Formato / Alinhamento	Descrição
rover_id	2	Texto, Alinhado à Direita (rjust)	Identificador único do rover (ex: " 1").
posicao_x	5	Texto, Alinhado à Direita (rjust)	Coordenada X com 1 casa decimal (ex: " 10.5").
posicao_y	5	Texto, Alinhado à Direita (rjust)	Coordenada Y com 1 casa decimal (ex: " 20.0").
bateria	5	Texto, Alinhado à Direita (rjust)	Nível de energia em percentagem (ex: " 99.9").
estado	12	Texto, Alinhado à Esquerda (ljust)	Estado operacional atual (ex: "em_missao ").
id_missao	10	Texto, Alinhado à Esquerda (ljust)	ID da missão ativa ou "Nenhuma" (ex: "M-001 ").

Tabela 5: Estrutura da mensagem de Telemetria

Detalhes dos Campos (Payload Implícito)

Aqui está o detalhe do que cada campo representa e os valores esperados:

Estado:

- "parado": Rover registado, imóvel e pronto para receber missões.
- "a_caminho": Rover em deslocação para as coordenadas do objetivo.
- "em_missao": Rover a executar a tarefa no local.
- "low_power_sl": (Low Power Sleep) Rover em hibernação para recarga de bateria.

Identificação da Missão (id_missao) Este campo fornece uma visão rápida do progresso sem necessidade de consultar o Mission Link:

- **Sem Missão:** O campo é preenchido com "Nenhuma...".
- **Com Missão:** O formato é composto pelo ID da missão seguido da percentagem de conclusão.
 - Formato: "{ID} {Progresso}%"
 - Exemplo: "1 45%....." (Missão 1 a 45% de progresso).

4.2. Implementação do Cliente (Rover)

O cliente de telemetria (telemetry_client.py) corre numa thread dedicada para não bloquear a operação principal do Rover.

4.2.1. Lógica de Envio e Formatação

A cada ciclo de envio (definido por SEND_INTERVAL = 5s), o cliente:

1. **Acede ao Estado Partilhado:** Obtém uma cópia segura (with lock) das variáveis globais do Rover (posição, bateria, estado).
2. **Formatação Rígida:** Converte os dados para *strings* e aplica *padding* (espaços ou zeros) para cumprir rigorosamente os tamanhos definidos na tabela acima.
 - *Exemplo:* Uma bateria de 95.5 ocupa 4 caracteres. O protocolo exige 5. O sistema adiciona um espaço à esquerda: " 95.5".
3. **Envio:** Envia o bloco de 39 bytes via socket.sendall().

4.2.2. Resiliência e Reconexão Automática

Para lidar com falhas de rede ou reinícios da Nave-Mãe, implementou-se uma lógica robusta de persistência:

- Se a conexão TCP cair, o cliente entra num ciclo de recuperação.
- Tenta reconectar até 5 vezes (MAX_RECONNECT_ATTEMPTS), com um intervalo de 10 segundos entre tentativas.

- Se a conexão for restabelecida, o fluxo de dados retoma automaticamente. Se o limite for excedido, o serviço desliga-se de forma graciosa para poupar recursos.

4.3. Implementação do Servidor (Nave-Mãe)

O servidor (`telemetry_server.py`) utiliza uma arquitetura Multi-threaded para conseguir monitorizar múltiplos Rovers simultaneamente.

1. **Socket Listener:** A *thread* principal aceita novas conexões na porta 50010.
2. **Worker Threads:** Para cada Rover que se conecta, é lançada uma *thread* dedicada (`handle_rover_telemetry`).
3. **Processamento:**
 - A *thread* lê continuamente blocos de 39 bytes.
 - Faz o *parsing* dos valores e deteta alertas (ex: se o estado contiver "low_power", emite um aviso de "BATERIA FRACA" nos logs).
 - Atualiza a base de dados em memória (`g_telemetry_database`), que é consumida pela API e pelo Mission Link.
4. **Persistência:** Além da memória RAM, cada atualização é registada num ficheiro histórico individual (`rover_X_history.json`) para análise posterior da trajetória e consumo energético.

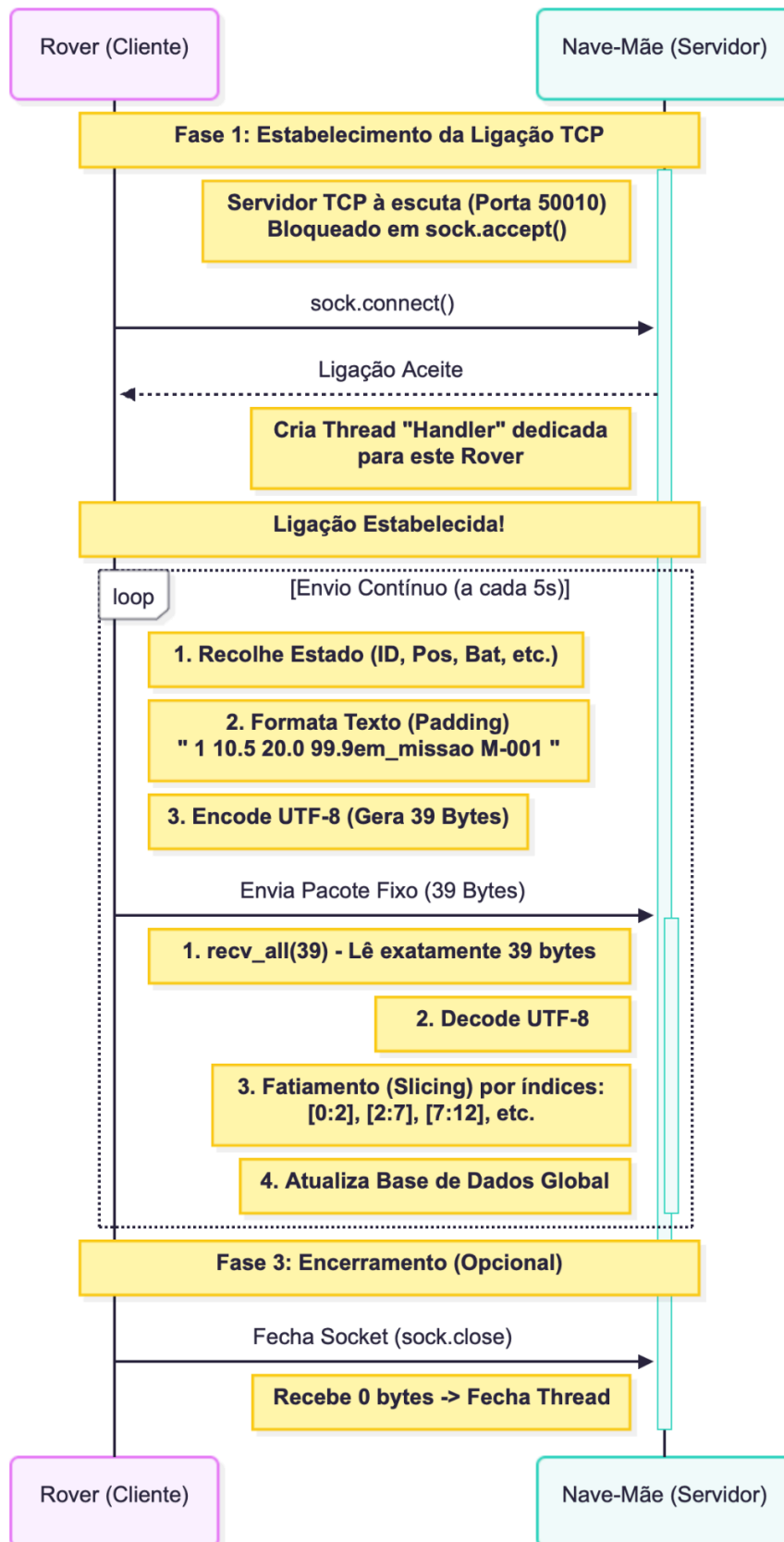


Figura 4: Diagrama de protocolo TCP

5. API de observação e ground control

Esta secção descreve de forma detalhada a camada de visualização desenvolvida para monitorizar os rovers e progresso de missões.

5.1 API de monitorização

Para possibilitar a observação externa, foi implementada uma API RESTful. A comunicação é estabelecida sobre o protocolo HTTP, operando na porta 8080, e utiliza JSON como formato de serialização de dados. A opção pelo HTTP, em detrimento de WebSockets, deveu-se à simplicidade de implementação e à facilidade de debugging proporcionada pelo modelo pedido-resposta.

A classificação da API como RESTful justifica-se pela adoção de uma arquitetura orientada a recursos (`/api/status`), pela utilização semântica do método HTTP GET para operações de leitura e pela natureza sem estado da comunicação.

É definido um endpoint único, acessível através de `"/api/status"`, desenhado para agregar o estado global do sistema numa única resposta. Ao processar um pedido GET neste endpoint, a Nave-Mãe compila uma cópia dos dados de todos os rovers ativos, bem como das filas de missões pendentes e concluídas. Para manter a consistência da informação num ambiente concorrente, o acesso às estruturas de dados em memória é protegido por locks (mecanismos de exclusão mútua), garantindo que a resposta reflete um estado válido do sistema no momento da consulta.

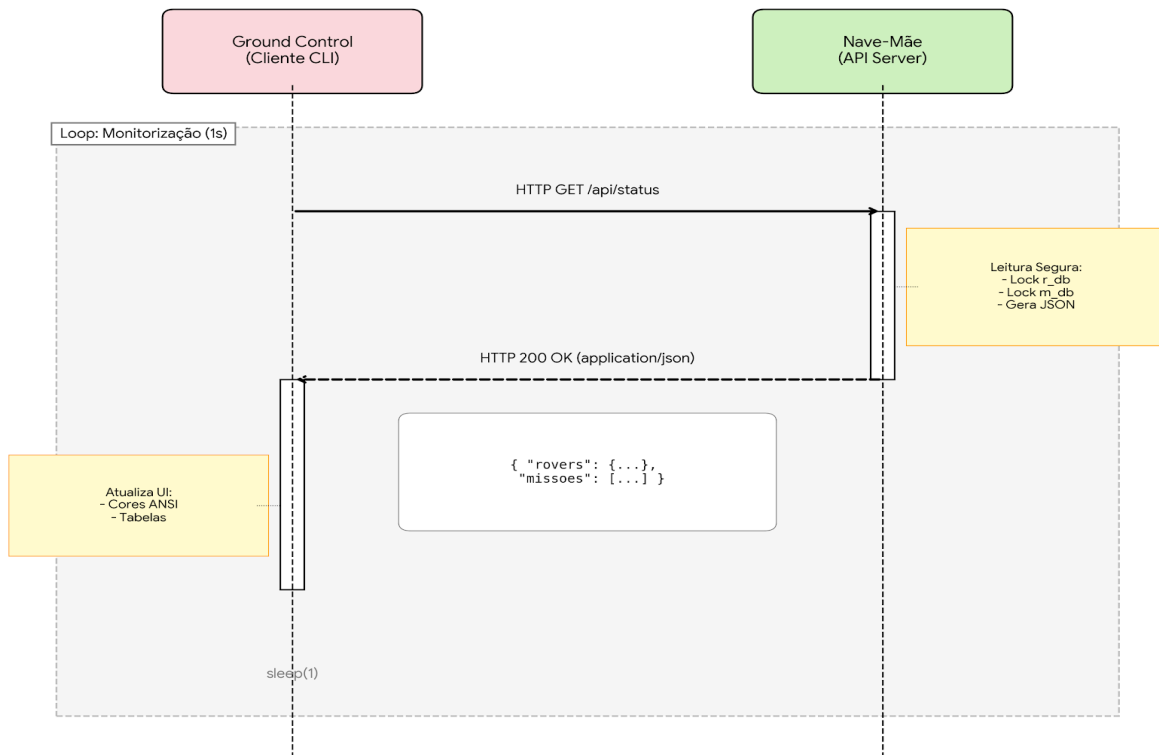


Figura 5: Diagrama de sequência da interação REST entre o Ground Control e a Nave-Mãe

Em seguida há um exemplo do formato da resposta fornecida pelo api ao pedido GET feito.

```

{
  "rovers": {
    "1": {
      "posicao": [45.2, 12.8],
      "bateria": 88.5,
      "estado_op": "em_missao",
      "missao_atual": "3"
    }
  },
  "missoes_pendentes": [
    "4", "5", "6", "7",
    "8", "9", "10"
  ],
  "missoes_concluidas": [
    { "id_missao": "1", "rover_id": "1", "tempo_exec": 30.5 },
    { "id_missao": "2", "rover_id": "1", "tempo_exec": 42.1 }
  ]
}

```

Figura 5: Exemplo de resposta fornecida

5.2 Ground Control

O componente Ground Control é materializado numa interface de linha de comandos (CLI) que atua como cliente da API de Monitorização.

O seu funcionamento baseia-se num mecanismo de polling, ou seja, um ciclo contínuo de monitorização que, com uma periodicidade de um segundo, envia pedidos HTTP à Nave-Mãe. Após o processamento da resposta, o cliente atualiza os dados de telemetria dos rovers e o estado das filas de missões, apresentando a informação no terminal em tempo real.

Ao nível visual, a prioridade foi garantir uma leitura imediata e intuitiva do estado da missão. Em vez de apresentar texto não formatado, a aplicação desenha uma tabela estruturada onde a cor é utilizada estrategicamente para guiar a atenção do operador. Graças a este sistema, a compreensão da situação da frota é instantânea, situações críticas, como níveis de bateria baixos, são evidenciadas a vermelho, enquanto as missões ativas são destacadas a amarelo.

Adicionalmente, a aplicação foi desenhada com foco na robustez. Caso a ligação à Nave-Mãe seja interrompida, a interface alerta imediatamente o utilizador sobre a perda de sinal e inicia um processo de reconexão automática em segundo plano, garantindo a recuperação do serviço sem necessidade de reinício manual por parte do operador.

5.2.1 Ground Control - Visual

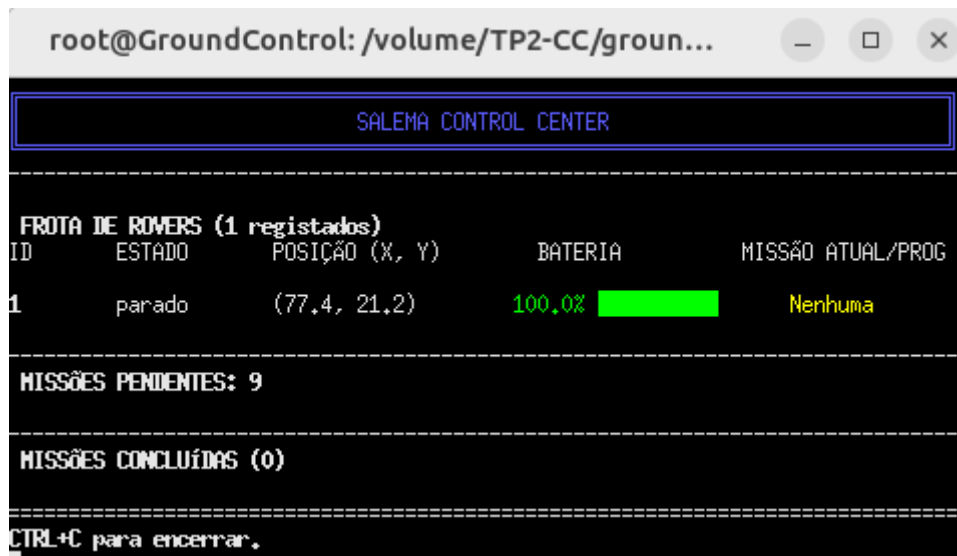


Figura 6: Inicialização do rover 1

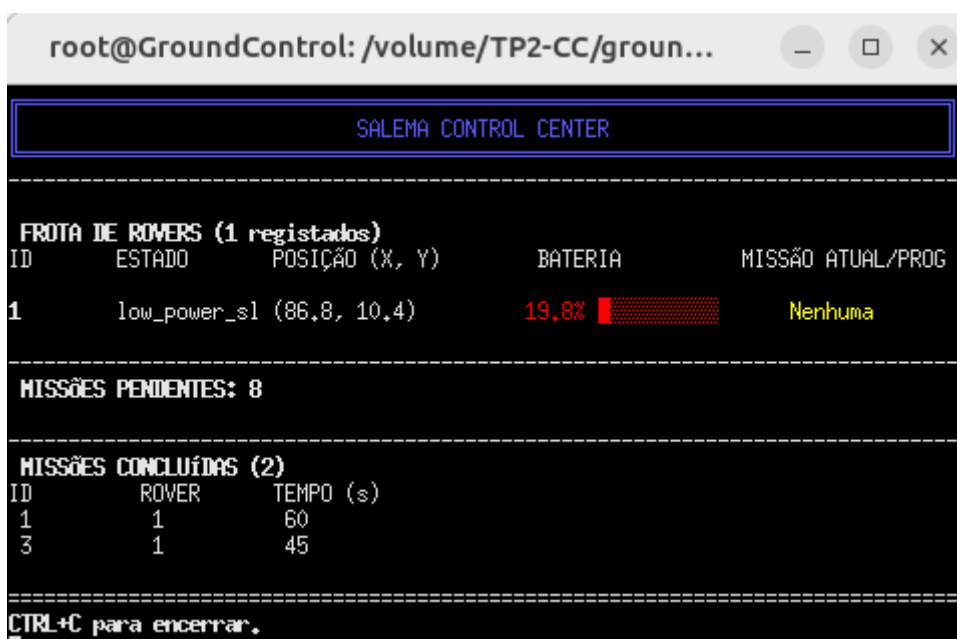


Figura 7: Estado crítico de bateria e missões concluídas

6. Testes realizados

6.1 Vários rovers em concorrência

Para validar a escalabilidade da arquitetura de múltiplas threads da Nave-Mãe, foi executado um teste de carga onde três rovers foram instanciados simultaneamente na topologia. O objetivo foi verificar se o servidor conseguia processar fluxos de telemetria (TCP) e pedidos de missão (UDP) concorrentes sem sofrer bloqueios ou corrupção de dados.

Como demonstrado na **Figura 8**, o sistema comportou-se de forma estável, todos os rovers conseguiram registrar-se com sucesso e a API de Observação refletiu corretamente os estados distintos de cada um ("em_missao", "a_caminho") em tempo real, confirmando que a gestão de threads e locks está corretamente implementada.

SALEMA CONTROL CENTER					

FROTA DE ROVERS (3 registrados)					
ID	ESTADO	POSIÇÃO (X, Y)	BATERIA	MISSÃO ATUAL/PROG	
1	em_missao	(23,0, 24,8)	51.0%		4 25%
2	em_missao	(52,3, 59,5)	53.0%		2 68%
3	a_caminho	(75,4, 24,2)	58.0%		6 0%

MISSÕES PENDENTES: 5					

MISSÕES CONCLUÍDAS (2)					
ID	ROVER	TEMPO (s)			
1	1	45			
3	3	40			
=====					
CTRL+C para encerrar.					

Figura 8: Rovers iniciados de forma concorrente

6.2 Perda de pacotes

Neste cenário, foi introduzida uma taxa de perda de pacotes de 20% nos links de comunicação entre os satélites e os rovers diretamente na topologia do CORE, simulando as condições degradadas de uma transmissão espacial real (**Figura 9**).

A análise dos logs (**Figura 10**) revela a atuação dos mecanismos de resiliência: observam-se múltiplas ocorrências de timeouts durante o envio de relatórios (mensagens assinaladas com socket timeout). No entanto, o sistema não falhou, os rovers detetaram a perda, aguardaram o tempo estipulado e retransmitiram a informação até obterem o ACK da Nave-Mãe, garantindo a integridade da missão sem intervenção externa.

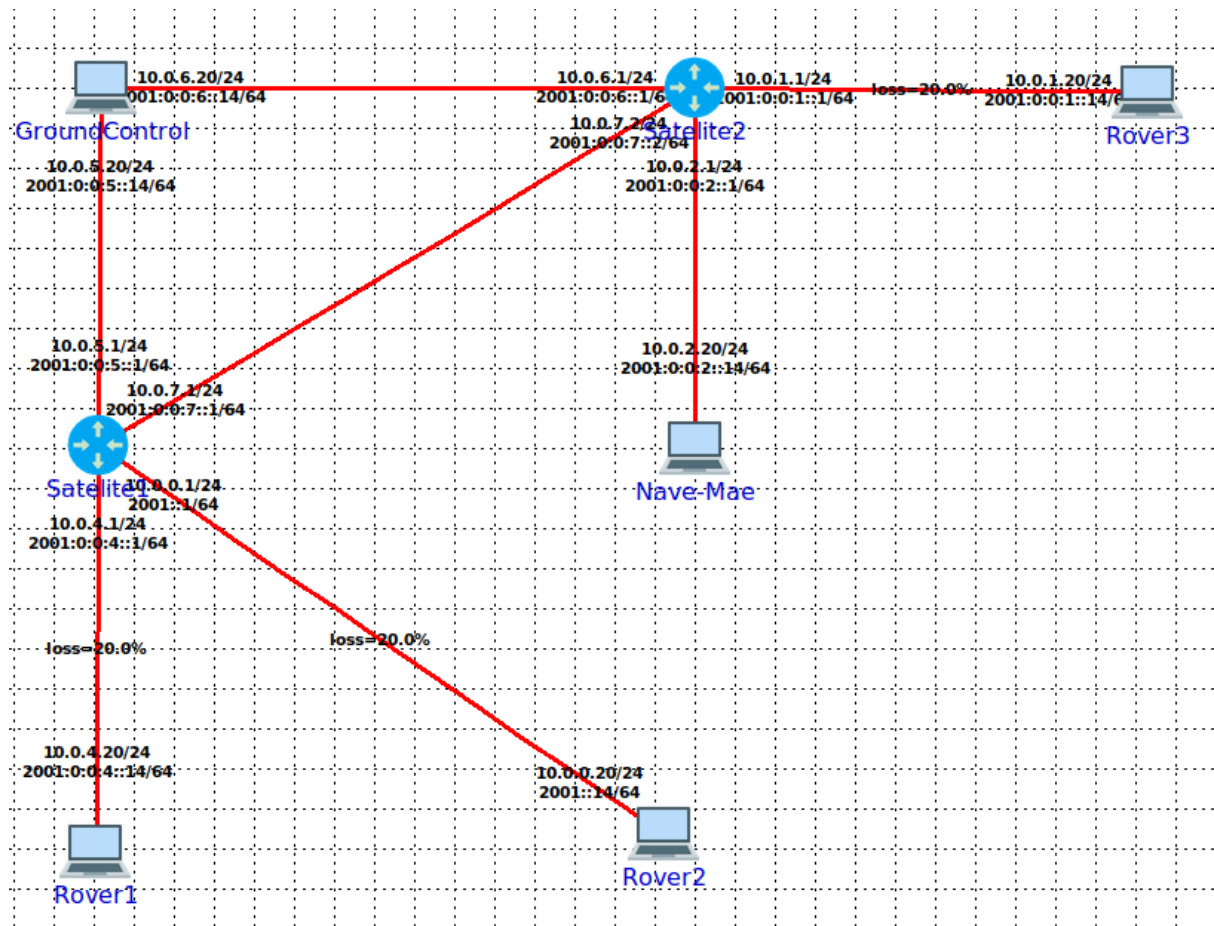


Figura 9: Topologia com 20% packet loss

```

2025-12-12 14:36:51,804 - MainThread - INFO - A simular Sincronização (UDP)...
2025-12-12 14:36:52,869 - Sync-01 - INFO - Nave Mae(10,0,2,20:50000) -> Rover 01
; sync ack
2025-12-12 14:36:52,869 - Main-01 - INFO - Sincronização (Simulada) bem sucedida
*
2025-12-12 14:36:52,869 - Telemetry-01 - INFO - Thread Telemetria iniciada (Prot
ocolo Fixo: 39 bytes).
2025-12-12 14:36:52,870 - Physics-01 - INFO - Simulador de Física iniciado. Limi
te de recarga: 20%.
2025-12-12 14:36:52,870 - MissionLink-01 - INFO - Serviço Mission Link iniciado
para Rover 01
2025-12-12 14:36:52,870 - Telemetry-01 - INFO - A ligar a 10,0,2,20:50010...
2025-12-12 14:36:52,870 - Main-01 - INFO - [Main] Serviço de Telemetria e Física
lançados.
2025-12-12 14:36:52,871 - MissionLink-01 - INFO - A pedir missao (Seq 1)...
2025-12-12 14:36:52,871 - Telemetry-01 - INFO - Ligado! Reiniciando contador de
falhas.
2025-12-12 14:36:52,872 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 1,2, 62,0)
|Bat=100,0|Est=parado|Mis=nenhuma
2025-12-12 14:36:54,873 - MissionLink-01 - WARNING - Socket timeout no envio de
MReq. Tentativas restantes: 2, a esperar 2s.
2025-12-12 14:36:56,876 - MissionLink-01 - INFO - Mae não deu missão (MAck). Esperar 10s.
2025-12-12 14:36:57,876 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 1,2, 62,0)|Bat= 99,0|Est=parado|Mis=nenhuma
2025-12-12 14:37:02,882 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 1,2, 62,0)|Bat= 98,0|Est=parado|Mis=nenhuma
2025-12-12 14:37:06,882 - MissionLink-01 - INFO - A pedir missao (Seq 2)...
2025-12-12 14:37:07,888 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 1,2, 62,0)|Bat= 97,0|Est=parado|Mis=nenhuma
2025-12-12 14:37:08,885 - MissionLink-01 - WARNING - Socket timeout no envio de MReq. Tentativas restantes: 2, a esperar 2s.
2025-12-12 14:37:12,890 - MissionLink-01 - WARNING - Socket timeout no envio de MReq. Tentativas restantes: 1, a esperar 4s.
2025-12-12 14:37:12,892 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 1,2, 62,0)|Bat= 96,0|Est=parado|Mis=nenhuma
2025-12-12 14:37:16,895 - MissionLink-01 - INFO - A viajar de (1,2, 62,0) para (10,0, 20,0)...
2025-12-12 14:37:17,928 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 2,3, 57,1)|Bat= 94,4|Est=a_caminho|Mis=1 0%

```

Figura 10: Logs do rover 1

6.3 Retransmissões

Para validar deterministicamente o algoritmo de Stop-and-Wait com Exponential Backoff, recorreu-se a um script de teste dedicado ("Chaos Server") programado para ignorar propositadamente as duas primeiras mensagens de qualquer sequência recebida.

Os resultados obtidos comprovam a robustez da lógica do cliente, a **Figura 11** mostra o servidor a descartar as tentativas iniciais, enquanto a **Figura 12** evidencia o comportamento do Rover, que após cada falha duplica o seu tempo de espera (2s, depois 4s) antes de tentar novamente. A comunicação é restabelecida com sucesso na terceira tentativa, validando a capacidade do protocolo MissionLink de recuperar de falhas persistentes na rede.

```

root@Nave-Mae:/volume/TP2-CC/testes# python3 teste2.py
2025-12-12 10:47:04,313 - [MÃE-CAOS] - --- SERVIDOR DE CAOS INICIADO NA PORTA 50001 ---
2025-12-12 10:47:04,314 - [MÃE-CAOS] - Vou ignorar as primeiras 2 mensagens de cada sequência para TESTAR RETRANSMISSÕES.

>> Recebido 'MReq' do Rover 1 (Seq 1). Esta é a tentativa #1.
2025-12-12 10:47:16,001 - [MÃE-CAOS] - [X] A IGNORAR (Simulação de Perda). O Rover deve retransmitir (Timeout).

>> Recebido 'MReq' do Rover 1 (Seq 1). Esta é a tentativa #2.
2025-12-12 10:47:20,005 - [MÃE-CAOS] - [X] A IGNORAR (Simulação de Perda). O Rover deve retransmitir (Timeout).

>> Recebido 'MReq' do Rover 1 (Seq 1). Esta é a tentativa #3.
2025-12-12 10:47:26,012 - [MÃE-CAOS] - [V] A RESPONDER (Sucesso). O Rover deve ficar feliz.

```

Figura 11: Nave-mãe CAOS - descarta os 2 primeiros pedidos

```

root@Rover1:/volume/TP2-CC/rover# python3 rover_main.py
2025-12-12 10:47:15,999 - MainThread - INFO - A simular Sincronização (UDP)...
2025-12-12 10:47:15,999 - Main-1 - INFO - Sincronização (Simulada) bem sucedida.
2025-12-12 10:47:16,000 - Telemetry-1 - INFO - Thread Telemetria iniciada (Proto
2025-12-12 10:47:16,000 - Physics-1 - INFO - Simulador de Física iniciado. Limit
2025-12-12 10:47:16,000 - Telemetry-1 - INFO - A ligar a 10.0.2.20:50010...
2025-12-12 10:47:16,000 - MissionLink-1 - INFO - Serviço Mission Link iniciado p
2025-12-12 10:47:16,000 - Main-1 - INFO - [Main] Serviço de Telemetria e Física
2025-12-12 10:47:16,001 - MissionLink-1 - INFO - A pedir missao (Seq 1)...
2025-12-12 10:47:16,001 - Telemetry-1 - WARNING - Erro na Telemetria ([Errno 111
2025-12-12 10:47:18,002 - MissionLink-1 - WARNING - Socket timeout no envio de M
2025-12-12 10:47:22,007 - MissionLink-1 - WARNING - Socket timeout no envio de M
2025-12-12 10:47:26,011 - Telemetry-1 - INFO - A ligar a 10.0.2.20:50010...
2025-12-12 10:47:26,012 - Telemetry-1 - WARNING - Erro na Telemetria ([Errno 111
2025-12-12 10:47:26,012 - MissionLink-1 - INFO - Mae não deu missão (MAck). Espe
2025-12-12 10:47:36,019 - MissionLink-1 - INFO - A pedir missao (Seq 2)...
2025-12-12 10:47:36,021 - Telemetry-1 - INFO - A ligar a 10.0.2.20:50010...
2025-12-12 10:47:36,021 - Telemetry-1 - WARNING - Erro na Telemetria ([Errno 111
2025-12-12 10:47:38,022 - MissionLink-1 - WARNING - Socket timeout no envio de M
Req. Tentativas restantes: 2, a esperar 2s.
2025-12-12 10:47:42,025 - MissionLink-1 - WARNING - Socket timeout no envio de M
Req. Tentativas restantes: 1, a esperar 4s.
2025-12-12 10:47:46,031 - MissionLink-1 - INFO - Mae não deu missão (MAck). Espe
rar 10s.

```

Figura 12: Logs do rover a retransmitir e finalmente obter resposta

6.4 Limites do Sistema e Tolerância a Falhas (50% Packet Loss)

Para determinar o ponto de rutura do sistema, a taxa de perda de pacotes na topologia CORE foi aumentada para níveis extremos de 50% (**Figura 13**). O objetivo deste teste foi verificar o comportamento do sistema quando a comunicação se torna impraticável.

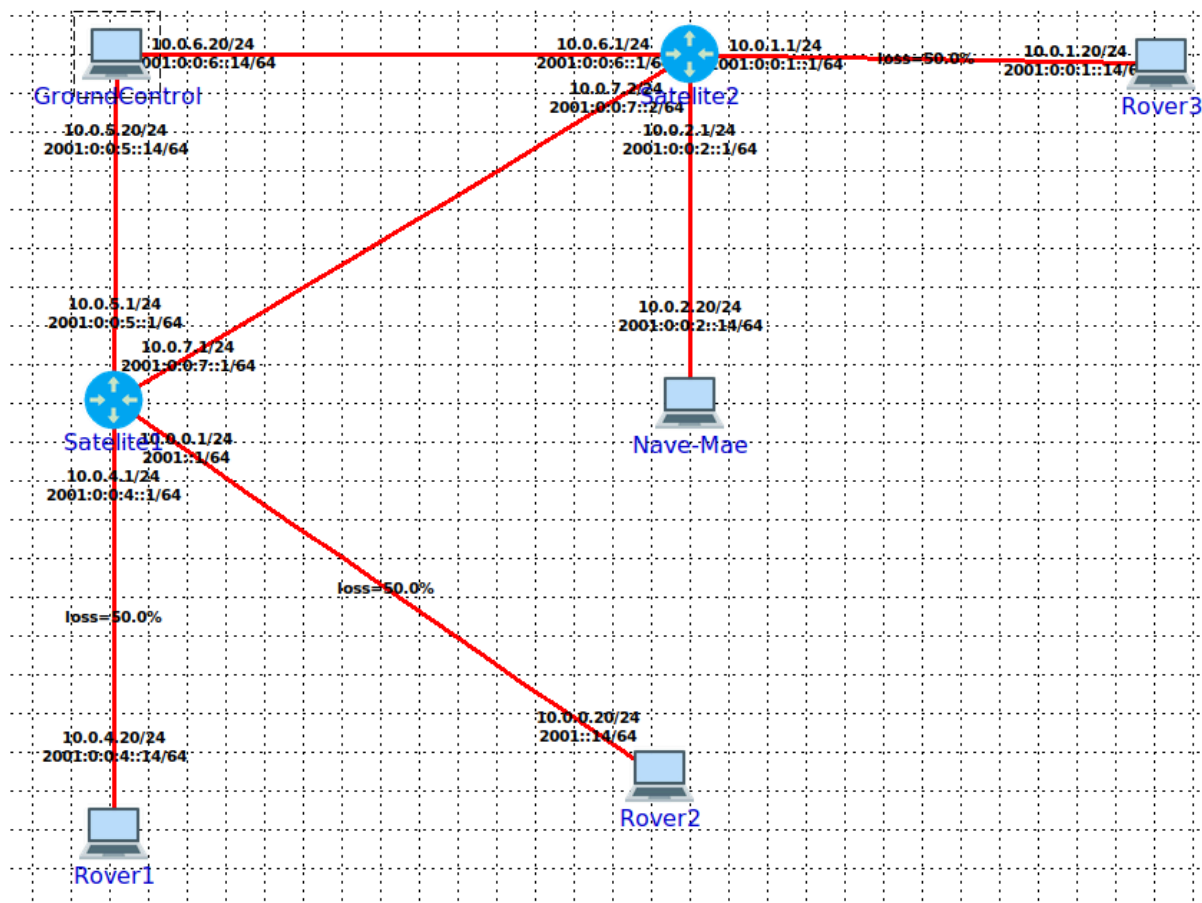


Figura 13 - Topologia com 50% de taxa de perda de pacotes

Na análise da **Figura 14**, temos que o Rover esgota frequentemente o limite de três tentativas de retransmissão, gerando erros de comunicação nos registros. Perante este cenário, o sistema exibe um comportamento de degradação positiva, isto é, ao falhar a confirmação de um relatório, o Rover não cancela a operação e prossegue com a missão física, garantindo assim que não fica paralisado por falta de rede.

Contudo, este desenho de autonomia acarreta um compromisso significativo ao nível da monitorização do Ground Control. Uma vez que o Rover continua a operar sem conseguir reportar o seu progresso, gera-se uma dessincronização de estado entre o Rover e a Nave-Mãe. Durante estes períodos de falha, a Nave-Mãe perde a consciência situacional do Rover, o que significa que, embora a missão física esteja a ser cumprida no terreno, o sistema central não tem forma de confirmar o sucesso ou intervir em caso de emergência até que a conectividade seja restabelecida.

Este comportamento evidencia a escolha de priorizar a disponibilidade do serviço em detrimento da consistência imediata da informação

```
2025-12-12 14:58:30,492 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 49,4|Est=em_missao|Mis=1 91%
2025-12-12 14:58:35,497 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 46,9|Est=em_missao|Mis=1 51%
2025-12-12 14:58:39,462 - MissionLink-01 - INFO - A enviar Relatório 68% (Seq 5)...
2025-12-12 14:58:40,503 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 44,4|Est=em_missao|Mis=1 68%
2025-12-12 14:58:41,465 - MissionLink-01 - WARNING - Socket timeout no envio de MRep. Tentativas restantes: 2, a esperar 2s.
2025-12-12 14:58:45,470 - MissionLink-01 - WARNING - Socket timeout no envio de MRep. Tentativas restantes: 1, a esperar 4s.
2025-12-12 14:58:45,507 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 41,9|Est=em_missao|Mis=1 68%
2025-12-12 14:58:50,512 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 39,4|Est=em_missao|Mis=1 68%
2025-12-12 14:58:51,473 - MissionLink-01 - WARNING - Socket timeout no envio de MRep. Tentativas restantes: 0, a esperar 8s.
2025-12-12 14:58:55,517 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 36,9|Est=em_missao|Mis=1 68%
2025-12-12 14:58:59,481 - MissionLink-01 - ERROR - Falha: a mae nao respondeu após 3 tentativas (seq 5).
2025-12-12 14:58:59,481 - MissionLink-01 - WARNING - Mãe nao confirmou relatório, continuando missao na mesma...
2025-12-12 14:59:00,522 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 34,4|Est=em_missao|Mis=1 68%
2025-12-12 14:59:05,527 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 31,9|Est=em_missao|Mis=1 68%
2025-12-12 14:59:09,491 - MissionLink-01 - INFO - A enviar Relatório 85% (Seq 6)...
2025-12-12 14:59:10,532 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 29,4|Est=em_missao|Mis=1 85%
2025-12-12 14:59:11,493 - MissionLink-01 - WARNING - Socket timeout no envio de MRep. Tentativas restantes: 2, a esperar 2s.
2025-12-12 14:59:15,497 - MissionLink-01 - WARNING - Socket timeout no envio de MRep. Tentativas restantes: 1, a esperar 4s.
2025-12-12 14:59:15,538 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 26,9|Est=em_missao|Mis=1 85%
2025-12-12 14:59:20,541 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 24,4|Est=em_missao|Mis=1 85%
2025-12-12 14:59:21,503 - MissionLink-01 - WARNING - Socket timeout no envio de MRep. Tentativas restantes: 0, a esperar 8s.
2025-12-12 14:59:25,547 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 21,9|Est=em_missao|Mis=1 85%
2025-12-12 14:59:29,512 - MissionLink-01 - ERROR - Falha: a mae nao respondeu após 3 tentativas (seq 6).
2025-12-12 14:59:29,512 - MissionLink-01 - WARNING - Mãe nao confirmou relatório, continuando missao na mesma...
2025-12-12 14:59:30,549 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 19,4|Est=em_missao|Mis=1 85%
2025-12-12 14:59:35,592 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 16,9|Est=em_missao|Mis=1 85%
2025-12-12 14:59:40,593 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 14,4|Est=em_missao|Mis=1 85%
2025-12-12 14:59:45,599 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 11,9|Est=em_missao|Mis=1 85%
2025-12-12 14:59:49,520 - MissionLink-01 - INFO - A enviar Conclusão 100% (Seq 7)...
2025-12-12 14:59:50,564 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 9,4|Est=em_missao|Mis=1 100%
2025-12-12 14:59:51,523 - MissionLink-01 - WARNING - Socket timeout no envio de MCon. Tentativas restantes: 2, a esperar 2s.
2025-12-12 14:59:55,528 - MissionLink-01 - WARNING - Socket timeout no envio de MCon. Tentativas restantes: 1, a esperar 4s.
2025-12-12 14:59:55,570 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 6,9|Est=em_missao|Mis=1 100%
2025-12-12 15:00:00,575 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 4,4|Est=em_missao|Mis=1 100%
2025-12-12 15:00:01,534 - MissionLink-01 - WARNING - Socket timeout no envio de MCon. Tentativas restantes: 0, a esperar 8s.
2025-12-12 15:00:05,581 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 1,9|Est=em_missao|Mis=1 100%
2025-12-12 15:00:09,536 - MissionLink-01 - ERROR - Falha: a mae nao respondeu após 3 tentativas (seq 7).
2025-12-12 15:00:09,537 - MissionLink-01 - WARNING - Mãe nao confirmou relatório de conclusao.
2025-12-12 15:00:09,537 - MissionLink-01 - INFO - A pedir missao (Seq 8)...
2025-12-12 15:00:09,564 - Physics-01 - WARNING - Bateria em 0,2%. Entrando em MODO DE RECARGA (30s).
2025-12-12 15:00:10,584 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 0,2|Est=low_power_sl|Mis=nenhuma
2025-12-12 15:00:11,539 - MissionLink-01 - WARNING - Socket timeout no envio de MReq. Tentativas restantes: 2, a esperar 2s.
2025-12-12 15:00:15,543 - MissionLink-01 - WARNING - Socket timeout no envio de MReq. Tentativas restantes: 1, a esperar 4s.
2025-12-12 15:00:15,590 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 0,2|Est=low_power_sl|Mis=nenhuma
2025-12-12 15:00:20,595 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 0,2|Est=low_power_sl|Mis=nenhuma
2025-12-12 15:00:21,545 - MissionLink-01 - WARNING - Socket timeout no envio de MReq. Tentativas restantes: 0, a esperar 8s.
2025-12-12 15:00:25,601 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 0,2|Est=low_power_sl|Mis=nenhuma
2025-12-12 15:00:29,550 - MissionLink-01 - ERROR - Falha: a mae nao respondeu após 3 tentativas (seq 8).
2025-12-12 15:00:29,551 - MissionLink-01 - WARNING - Sem resposta da mae, a esperar 5s...
2025-12-12 15:00:30,606 - Telemetry-01 - INFO - Enviado: ID=01|Pos=( 14,0, 22,5)|Bat= 0,2|Est=low_power_sl|Mis=nenhuma
```

Figura 14: Logs de falha do rover

7. Conclusão

Este projeto permitiu desenvolver com sucesso um sistema de comunicação distribuído completo, simulando a interação entre uma Nave-Mãe e vários Rovers autônomos. O principal objetivo foi alcançado: criar uma arquitetura capaz de gerir missões críticas e monitorização em tempo real, lidando corretamente com as limitações de rede. Ao longo do desenvolvimento, destacam-se três pontos principais:

Fiabilidade sobre UDP: A implementação manual de ACKs, timeouts e retransmissões no MissionLink provou que é possível construir um protocolo fiável sobre um canal inseguro, garantindo que nenhuma missão é perdida.

Eficiência do TCP e Largura Fixa: A Telemetria demonstrou a utilidade do TCP para fluxos contínuos de dados. A decisão de usar mensagens de tamanho fixo (39 bytes) simplificou muito o processamento e evitou erros de leitura.

Robustez e Concorrência: O uso de threads e locks permitiu que o Rover simulasse a física, comunicasse e gerisse a sua bateria simultaneamente, sem bloquear.

Em suma, o sistema final é funcional e resiliente. Os Rovers conseguem operar de forma autónoma, recuperar de falhas de conexão e gerir a sua energia, cumprindo todos os requisitos propostos na unidade curricular.